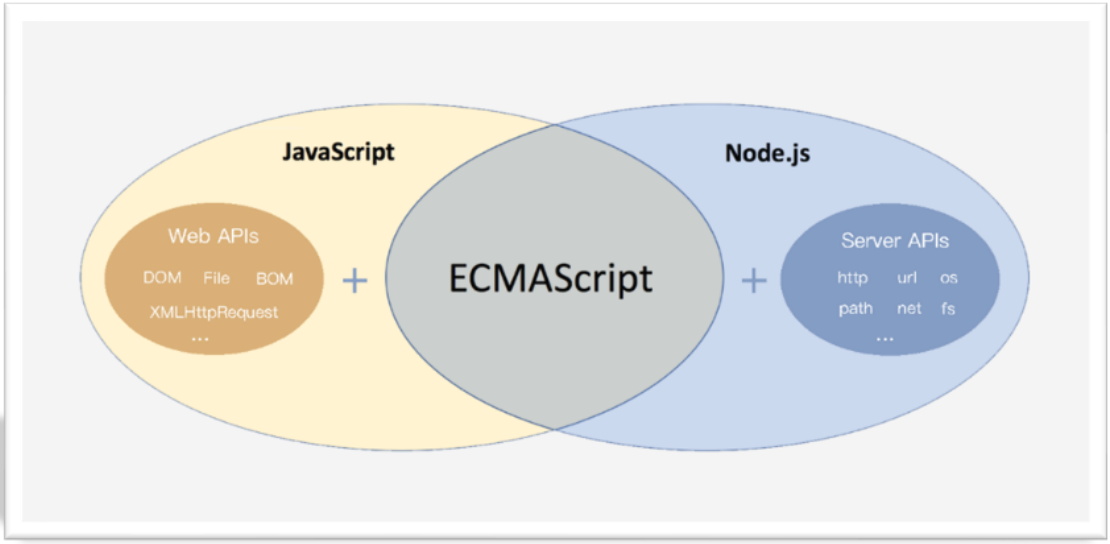
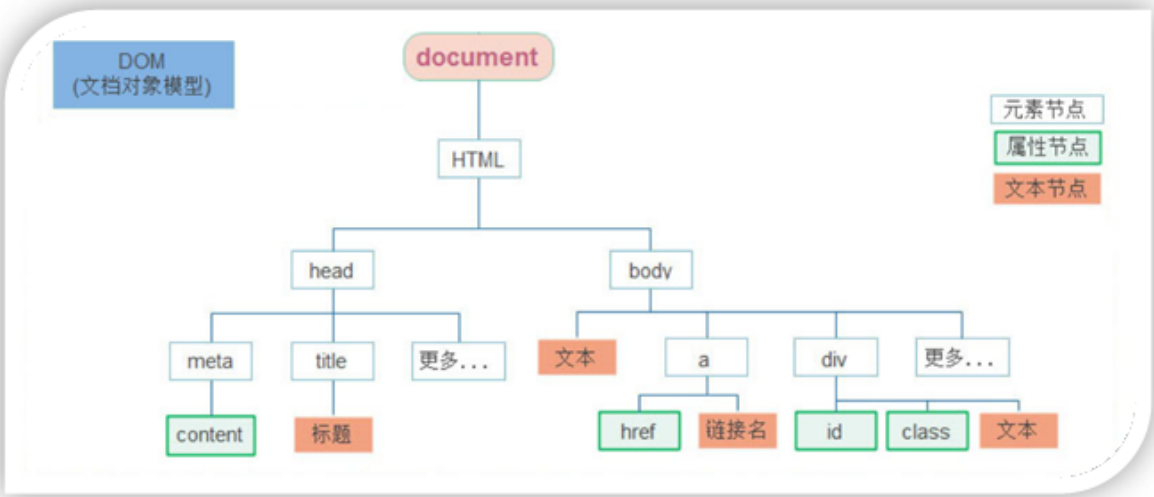
- **JavaScript与ECMAScript**
  - ECMAScript 是一套语法标准
    - 简称ES
  - 我们之前学的语法其实就是 ECMAScript 里的语法
    - 变量、数据类型、运算等规则都是 ECMAScript 规定的
  - JavaScript 是什么?
    - 遵守 ECMAScript 规则的一套编程语言
    - 严格来讲，ECMAScript 配合浏览器提供的 Web APIs 才称之为 JavaScript
  - Web APIs 是什么?
    - 浏览器提供的一套操作浏览器、页面内容的功能（主要是一些对象和方法）
  - JavaScript和node.js



- **DOM是什么**
  - DOM是浏览器提供的一套专门用来操作网页内容的功能
  - DOM的核心思想
    - 把网页内容当做对象来处理
  - DOM作用
    - 开发网页内容特效和实现用户交互
  - DOM全称
    - Document Object Model（文档对象模型）

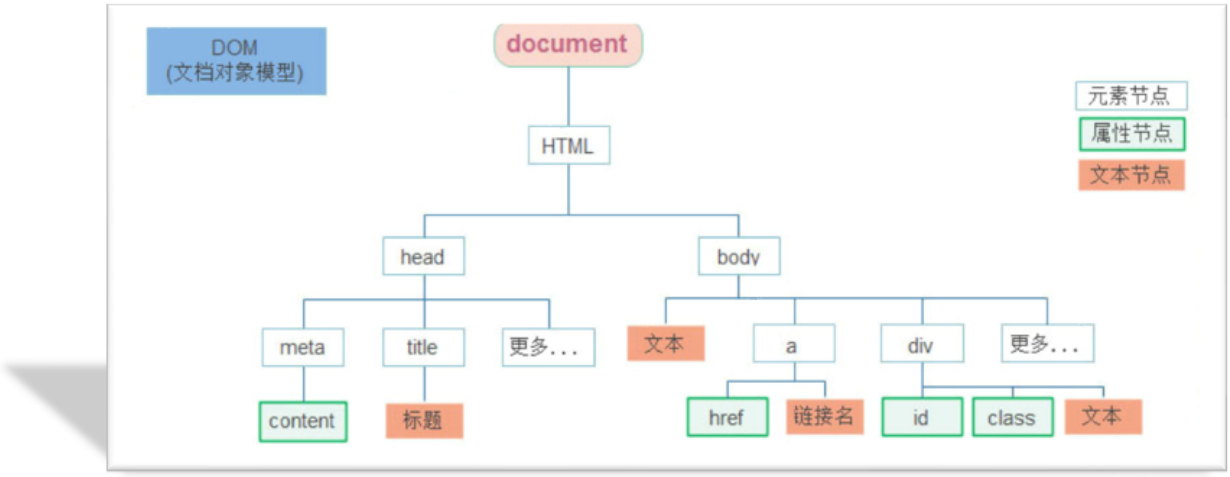
- **DOM树**
  - 将 HTML 文档以树状结构直观的表现出来，我们称之为文档树或 DOM 树
  - 描述网页内容关系的名词
  - 作用：文档树直观的体现了标签与标签之间的关系

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>标题</title>
</head>
<body>
  文本
  <a href="">链接名</a>
  <div id="" class="">文本</div>
</body>
</html>
```

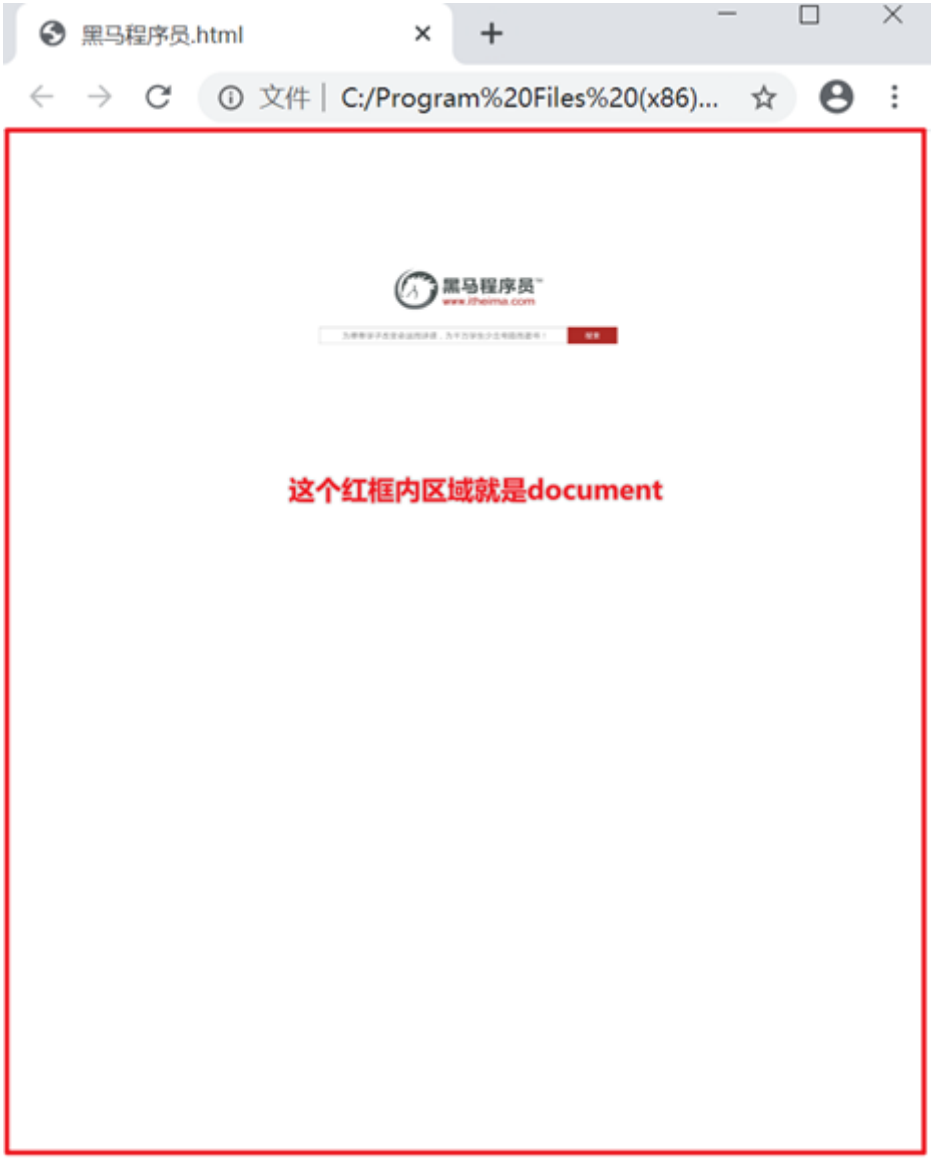


- **DOM节点**
  - DOM节点
    - DOM树里每一个内容都称之为节点
  - 节点类型
    - 元素节点
      - 所有的标签 比如 body、 div
      - html 是根节点

    - 属性节点
        - 所有的属性 比如 href
    - 文本节点
        - 所有的文本
    - 其他



- **document**
    - 是 DOM 里提供的一个对象
    - 所以它提供的属性和方法都是用来访问和操作网页内容的
        - 例：document.write()
    - 代表浏览器显示网页内容的区域
    - 网页所有内容都在document里面
    - document 是学习 DOM 的核心



- **查找元素节点**
  - 根据 id 来查找dom元素节点
    - 语法：

```
document.getElementById('标签的id')
```

- 根据id查找标签
- 传入的id是字符串，记得加引号，直接写id名即可，不需要加 #
- 返回一个匹配到 ID 的 DOM Element 对象（所有节点都是对象）
- 找不到会得到null
- 可以通过对象里面的 nodeType 属性来标识节点类型

```
<!-- 上面是标签 -->
<div id="nav"></div>
<script>
    // 查找 id为nav的标签
    let nav = document.getElementById('nav')
    console.log(nav)
</script>
```

- 查找html和body元素节点
  - 查找 html 元素

```
document.documentElement
```

  - 查找 body 元素

```
document.body
```

- **操作元素属性**
  - 直接修改元素的属性
    - 语法：

```
对象.属性 = 值
```

```
// 1. 获取 img 对应的 DOM 元素
let pic = document.getElementById('pic')
// 2. 修改属性
pic.src = './images/lion.webp'
pic.width = 400
pic.alt = '图片不见了...'
```

  - 通过 setAttribute 方法修改
    - 语法：

```
对象.setAttribute('属性', '值')
```

```
// 1. 获取 img 对应的 DOM 元素
let pic = document.getElementById('pic')
// 2. 修改属性
pic.setAttribute('src', './images/potala.webp')
pic.setAttribute('width', '540')
```

注意：

1. 属性和值都要加引号
2. 目前两种方式没啥区别
3. 更推荐使用对象.属性方式

  - 修改元素的样式

- 语法：

```
对象.style.样式属性 = 值
```

```
// 获取 DOM 节点
let box = document.getElementById('box');
box.style.color = 'red';
box.style.width = '300px';
// css 属性的 - 连接符与 JavaScript 的 减运算符
// 冲突，所以要改成驼峰法
box.style.backgroundColor = 'pink';
```

4

注意：

1. 修改样式通过style属性引出
2. 如果属性有-连接符，需要转换为小驼峰命名法
3. 赋值的时候，需要的时候不要忘记加css单位

- **操作元素文本**
  - document.write
    - 只能将文本内容追加到 </body> 前面的位置
    - 文本中包含的标签会被解析

```
// 永远都只是追加操作，且只能位置 </body> 前
document.write('Hello World!');
document.write('<h3>你好，世界！</h3>')
```

  - innerText 属性
    - 将文本内容添加/更新到任意标签位置
    - 文本中包含的标签不会被解析

```
// innerText 将文本内容添加 / 更新到任意标签位置
let info = document.getElementById('info')
// intro.innerText = '嗨~ 我叫李雷！'
info.innerText = '<h4>嗨~ 我叫李雷！</h4>'
```

  - innerHTML 属性
    - 将文本内容添加/更新到任意标签位置
    - 文本中包含的标签会被解析

```
// 2. innerHTML属性
box.innerHTML = '<h3>前端程序员<br>的头发都很多</h3>'
```

- **时间对象实例化**
  - 在代码中发现了 new 关键字时，一般将这个操作称为实例化
  - 创建一个时间对象并获取时间
    - 获得当前时间

```
let date = new Date()
```

- 获得指定时间

```
let date = new Date('1949-10-01')
```

- 时间对象：用来表示时间的对象
- 作用：可以得到当前系统时间

- **时间对象方法**
  - 因为时间对象返回的数据我们不能直接使用，所以需要转换为实际开发中常用的格式

方法	作用	说明
getFullYear()	获得年份	获取四位年份
getMonth()	获得月份	取值为 0 ~ 11
getDate()	获取月份中的每一天	不同月份取值也不相同
getDay()	获取星期	取值为 0 ~ 6
getHours()	获取小时	取值为 0 ~ 23
getMinutes()	获取分钟	取值为 0 ~ 59
getSeconds()	获取秒	取值为 0 ~ 59

- **时间戳**
  - 什么是时间戳
    - 是指1970年01月01日00时00分00秒起至现在的毫秒数，它是一种特殊的计量时间的方式
  - 三种方式获取时间戳
    - 使用 getTime() 方法

```
// 1. 实例化
let date = new Date()
// 2. 获取时间戳
console.log(date.getTime())
```

- 简写 +new Date()

```
console.log(+new Date())
```

- 使用 Date().now()

```
console.log(Date.now())
```

- 无需实例化
- 但是只能得到当前的时间戳， 而前面两种可以返回指定时间的时间戳

- **定时器函数使用**
  - 开启定时器

```
setInterval(函数, 间隔时间)
```

- 作用：每隔一段时间调用这个函数
- 间隔时间单位是毫秒

```
function repeat() {
    console.log('前端程序员，就是头发多咋滴~~')
}
// 每隔一秒调用repeat函数
setInterval(repeat, 1000)
```

注意：函数名字**不需要加括号**

- 关闭定时器

```
let 变量名 = setInterval(函数, 间隔时间)
clearInterval(变量名)
```

- 一般不会刚创建就停止，而是满足一定条件再停止